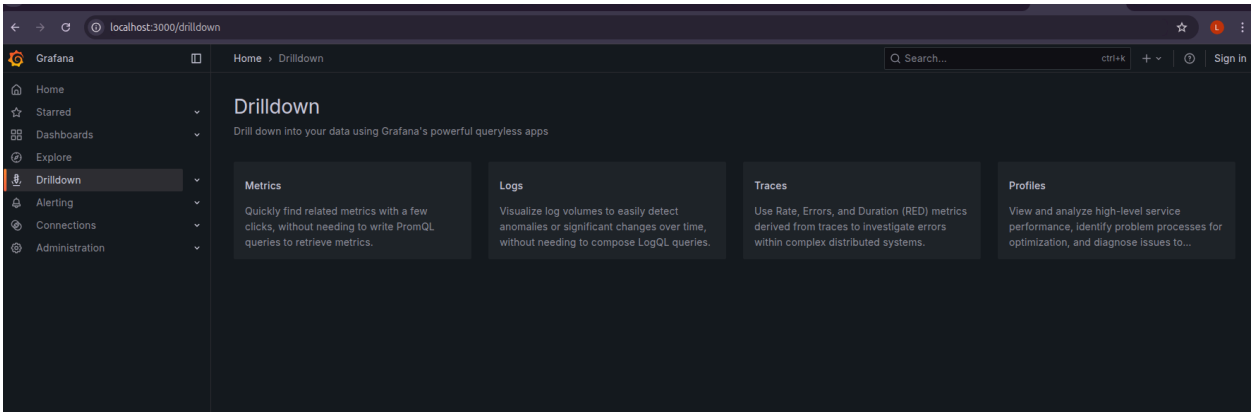
Explanation

Access Grafana on port 3000 and choose drilldown to see the availale traces.

Click on traces.



Notice we already have the datasource as “Tempo” selected and we are in “Root spans section”

No traces available because we did not do anything on the application.

What is a Root Span?

> A root span is the first span of a trace — it has no parent.

It represents:

- The **entry point** of a request into your system

Common examples of root spans

- Incoming HTTP request (`GET /api/orders`)
- Message consumed from Kafka
- Cron job execution
- Background worker start

In our app

This span is the root span:

```
quickpizza :static  (GET200)
```

Because:

- It started the trace
- Nothing called it
- It has **no parent span**

What are All Spans?

> All spans = every span in the trace, including root and child spans.

This includes:

- Root span
- Child spans
- Grandchild spans

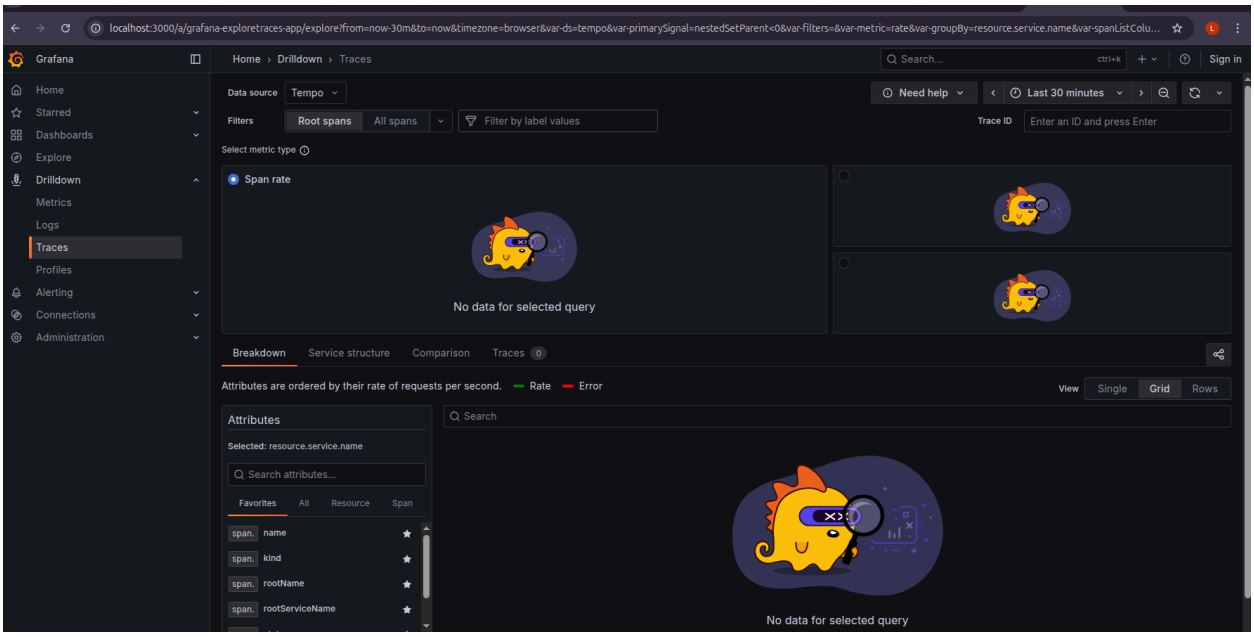
Example trace (monolith with instrumentation)

```
Root span:
HTTPGET/order

Child spans:
 ├── validate-user
 ├──create-order
 └── db.insert
```

Here:

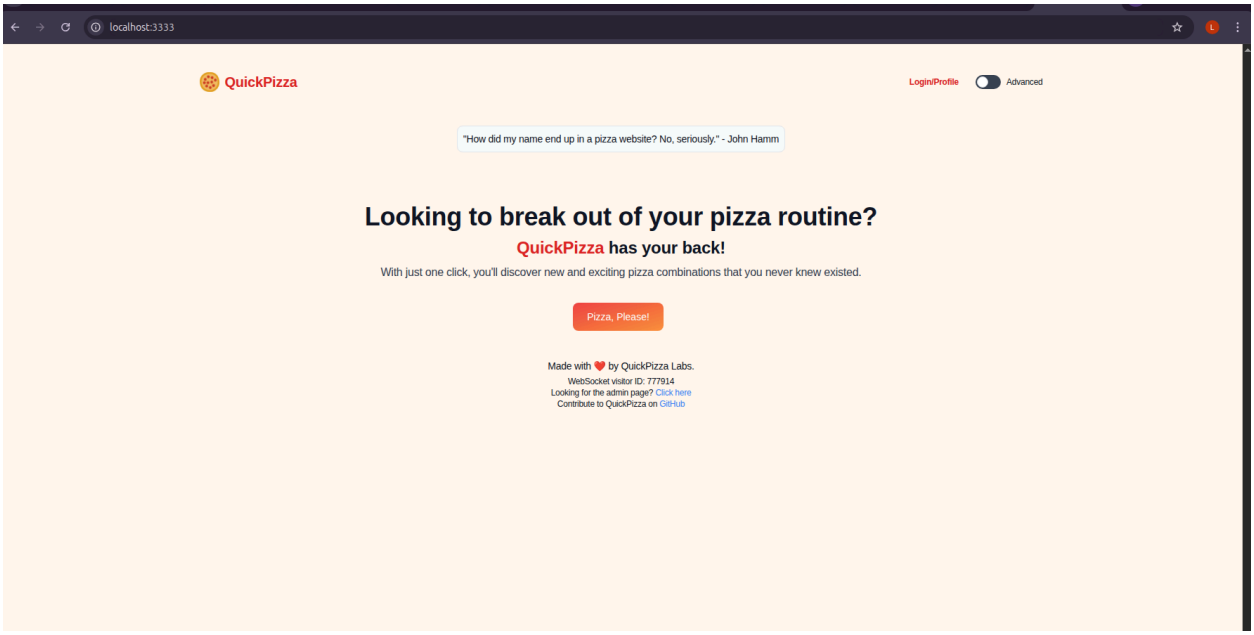
- **Root spans view** → shows only `HTTP GET /order`
- **All spans view** → shows all 4 spans

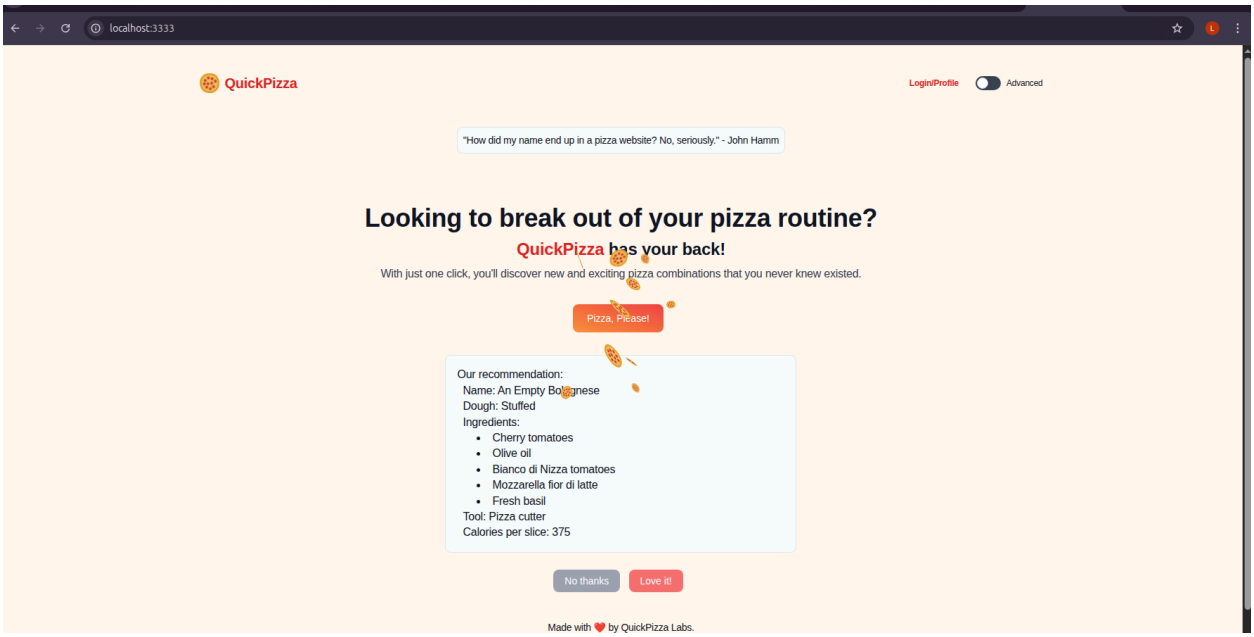


Lets add some traffic to our app.

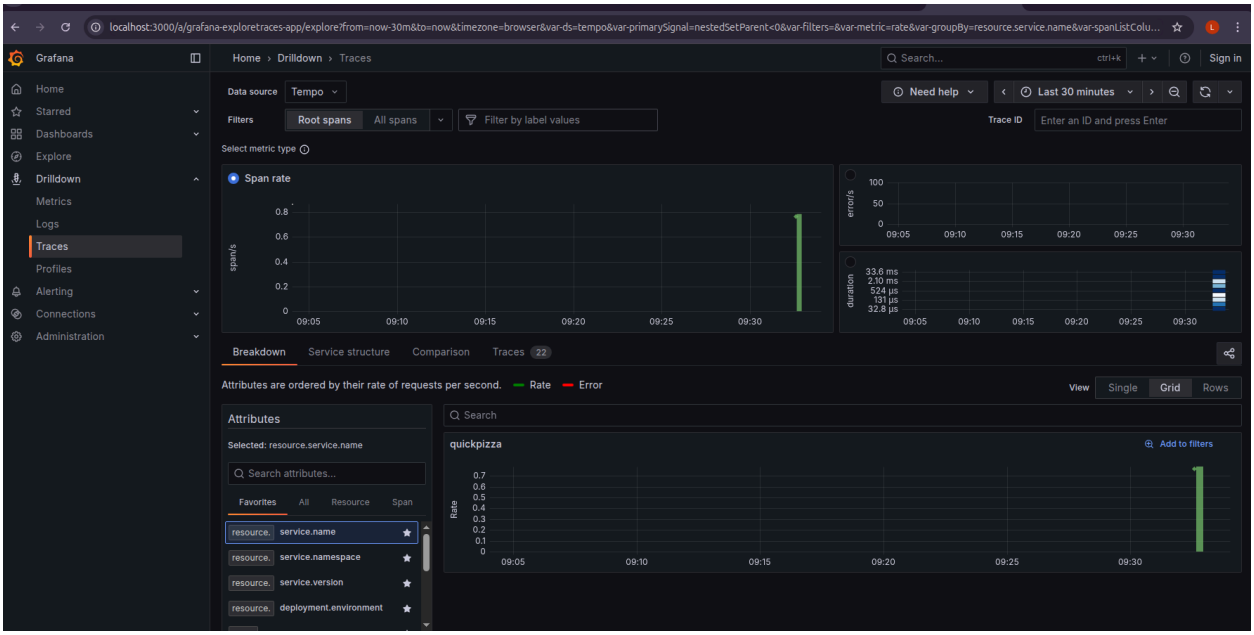
Access it with port 3333.

Refresh or Click on “Pizza,Please!” button multiple times.





Now come back to our grafana UI, and you will see some traces generated here.



You can see above one span.

💡 As we already know

A **trace** represents **one single request** as it travels through your system (for example: *"User orders a pizza"*).
That trace is broken into **smaller steps**, and **each step is called a span**.

What is a span?

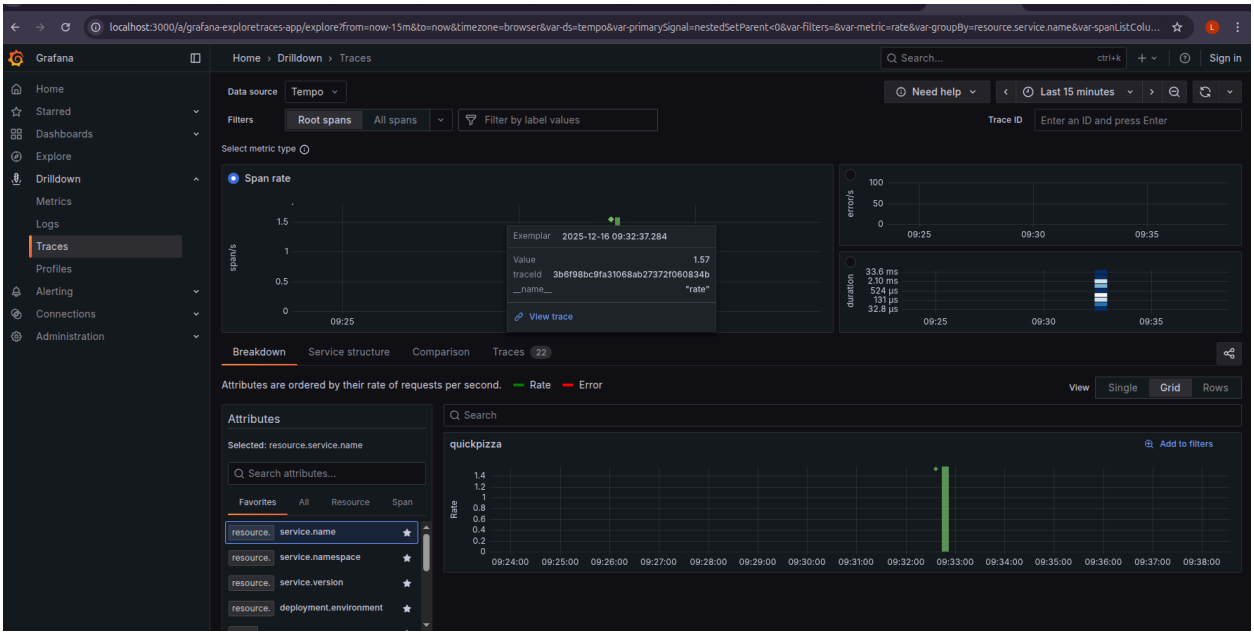
👉 A **span** is **one unit of work** inside a trace.
Think of a span as:

- **One operation**
- **One function call**
- **One service doing something**

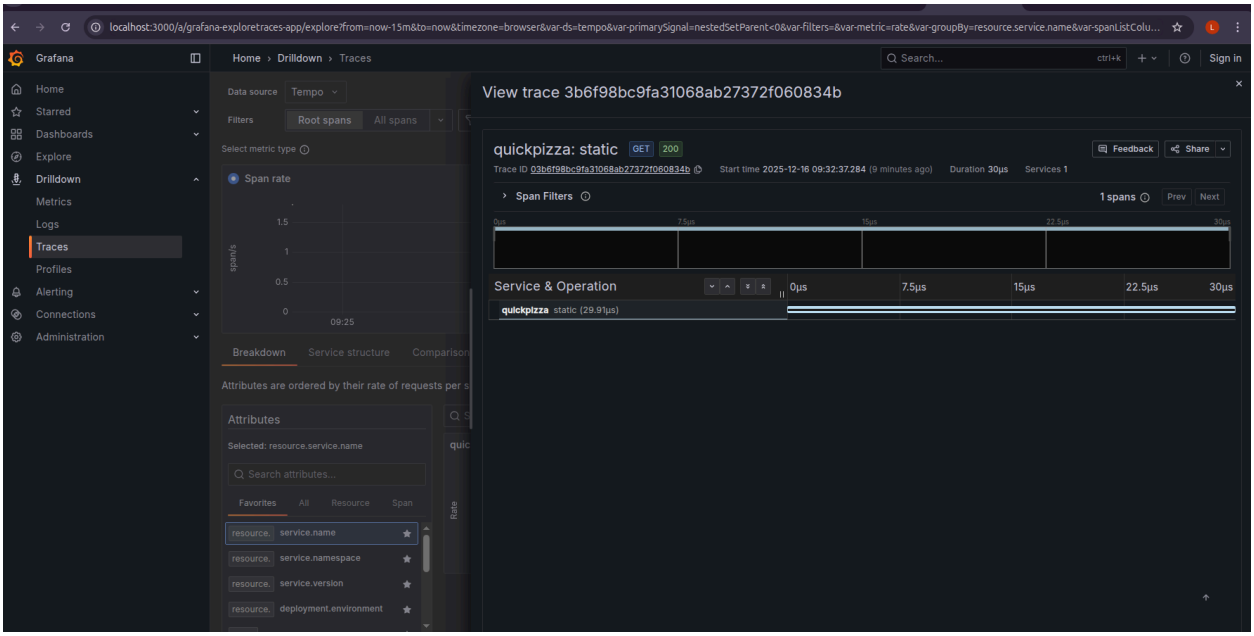
In simple words:

> Span = one step of a request

Hover over the span and then click on view traces



You will see the details about the span.



This trace represents **one HTTP request**:

```
GET/static →200 OK
```

In this case, the request produced **only one span**, so:

> 1 request = 1 trace = 1 span
>
> *(because only the entry-point was instrumented)*

Service & Operation

```
quickpizza :staticGET200
```

- **Service**: `quickpizza`
- **Span name**: `static`
- **Method**: `GET`
- **Status**: `200 OK`

This span represents the **entry point** of the request.

Trace ID

A **Trace ID** uniquely identifies the request across the system.

All spans created while handling this request would share the same Trace ID.

Timing

- **Duration**: ~30 µs
- Indicates a very fast, in-memory/static response

Why only one span?

Only **HTTP server auto-instrumentation** was active:

- No manual spans
- No DB or internal HTTP calls instrumented

So only the **root span** exists.

How spans are created (Root → Child)

When a request enters the system, the tracer **first creates a root span**

(for example, the incoming HTTP request).

```
GET/static   ← root span
```

As the request is processed, **child spans are created** whenever:

- Internal HTTP calls are made
- Database queries are executed
- Business logic is manually instrumented

These child spans are automatically linked to the root span using the **trace context**:

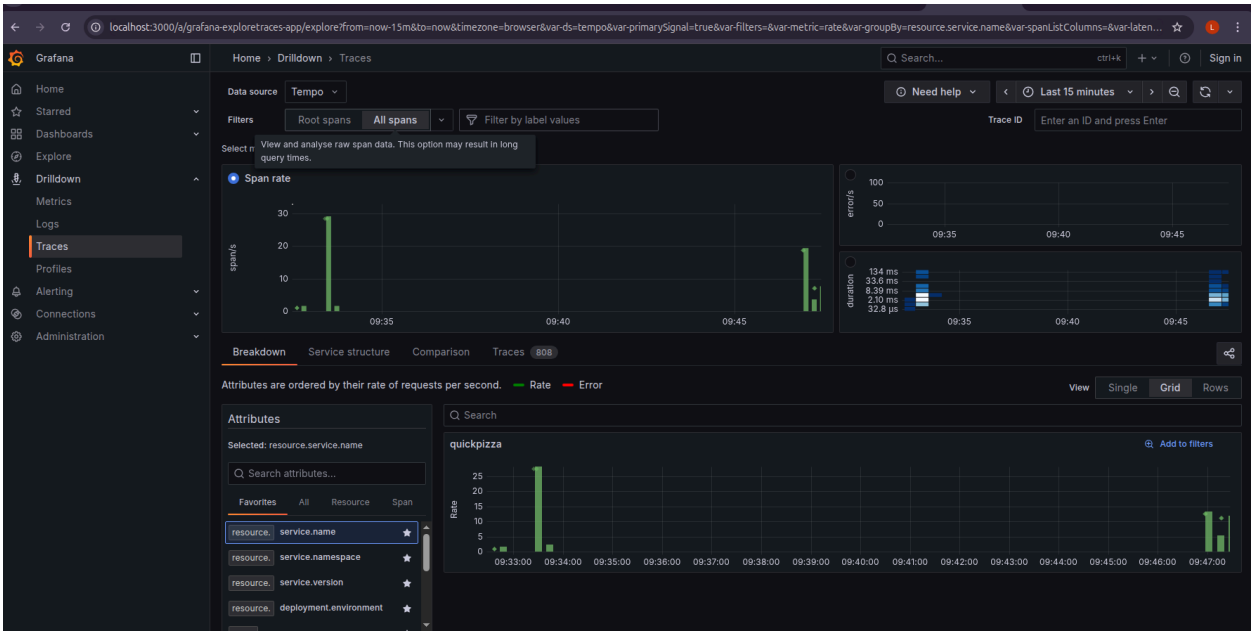
```
Rootspan
 ├── Childspan (HTTP call)
 ├── Childspan (DB query)
 └── Childspan (business logic)
```

Initially, when viewing **Root spans**, only the entry-point span is visible.

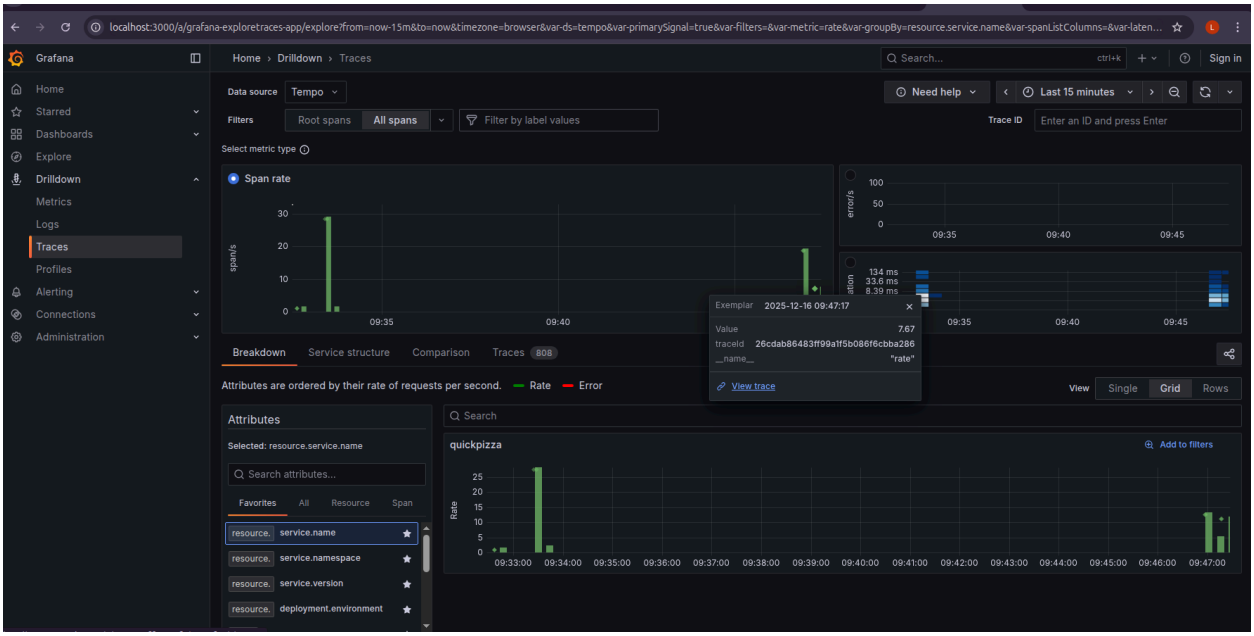
Later, when switching to **All spans**, the full parent–child hierarchy becomes visible, showing how the request actually behaved internally.

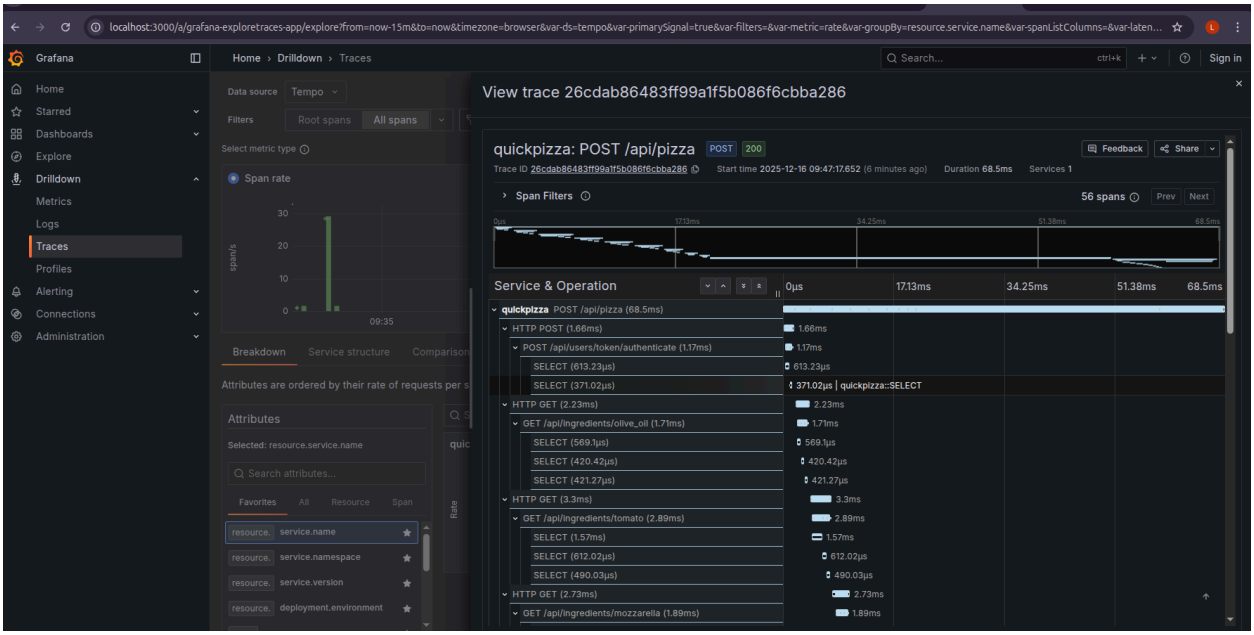
On the above step we saw the root span, Now go back to our application and hit more requests. Come back to Grafana Dashboard. And choose all spans.

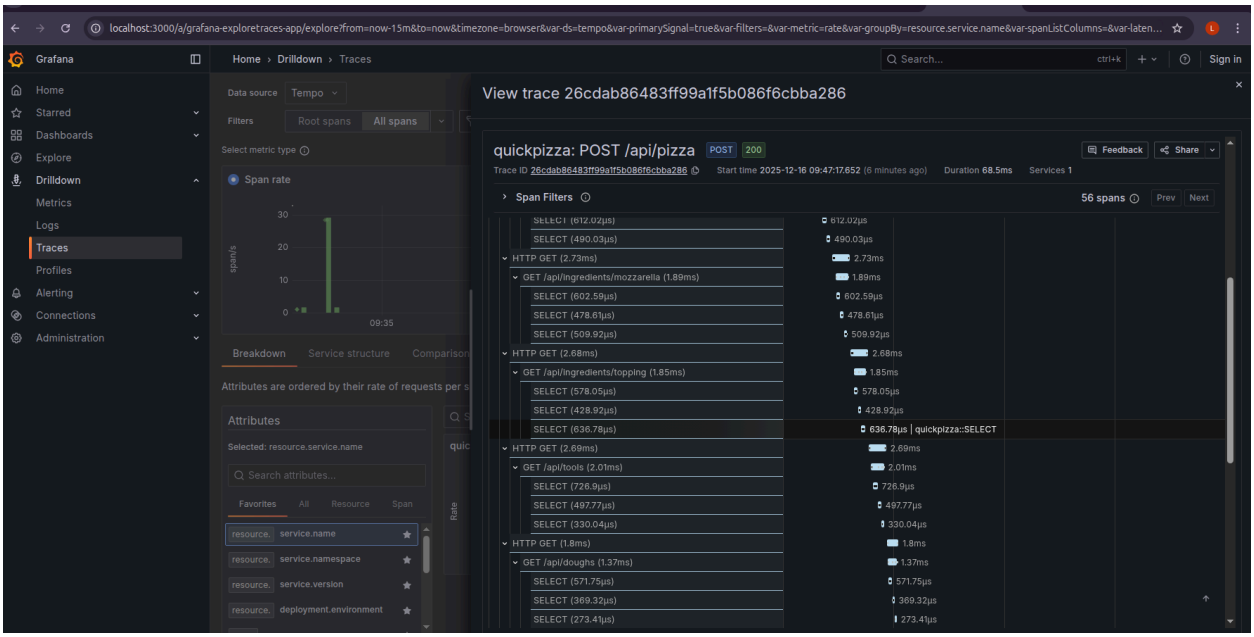
You can see more span generated.

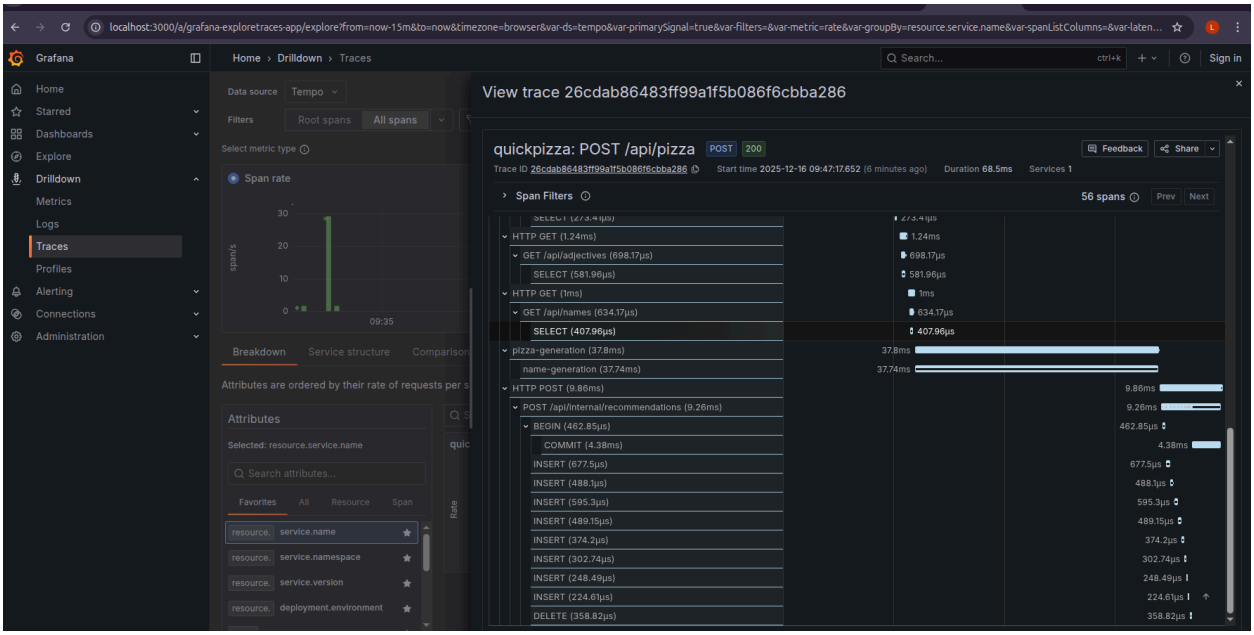


Now Hover over a child span and click on view trace.




localhost:3000/a/grafana-exploretraces-app/explore?from=now-15m&to=now&timezone=browser&var-ds=tempo&var-primarySignal=true&var-filters=&var-metric=rate&var-groupBy=resource.service.name&var-spanListColumns=&var-laten...
Grafana
Home › Drilldown › Traces
Search...
Sign in
Home
Starred
Dashboards
Explore
Drilldown
Metrics
Logs
Traces
Profiles
Alerting
Connections
Administration
Data source
Tempo
Filters
Root spans
All spans
Select metric type
Span rate
Breakdown
Service structure
Attributes
Selected: resource.service.name
Search attributes...
Favorites
All
Resource
Span
resource. service.name
resource. service.namespace
resource. service.version
resource. deployment.environment
View trace 26cdab86483ff99a1f5b086f6cbba286
quickpizza: POST /api/pizza
POST
200
Feedback
Share
Trace ID 26cdab86483ff99a1f5b086f6cbba286
Start time 2025-12-16 09:47:17.652 (6 minutes ago)
Duration 68.5ms
Services 1
Span Filters
56 spans
Prev
Next
Service & Operation
0µs
17.13ms
34.25ms
51.38ms
68.5ms
quickpizza POST /api/pizza (68.5ms)
HTTP POST (1.66ms)
POST /api/users/token/authenticate (1.17ms)
SELECT (613.23µs)
SELECT (371.02µs)
HTTP GET (2.23ms)
GET /api/ingredients/olive_oil (1.71ms)
SELECT (569.1µs)
SELECT (420.42µs)
SELECT (421.27µs)
HTTP GET (3.3ms)
GET /api/ingredients/tomato (2.89ms)
SELECT (1.57ms)
SELECT (612.02µs)
SELECT (490.03µs)
HTTP GET (2.73ms)
GET /api/ingredients/mozzarella (1.89ms)


localhost:3000/a/grafana-exploretraces-app/explore?from=now-15m&to=now&timezone=browser&var-ds=tempo&var-primarySignal=true&var-filters=&var-metric=rate&var-groupBy=resource.service.name&var-spanListColumns=&var-laten...
Grafana
Home › Drilldown › Traces
Search...
Sign in
View trace 26cdab86483ff99a1f5b086f6cbba286
quickpizza: POST /api/pizza
POST
200
Feedback
Share
Trace ID 26cdab86483ff99a1f5b086f6cbba286
Start time 2025-12-16 09:47:17.652 (6 minutes ago)
Duration 68.5ms
Services 1
Span Filters
56 spans
Prev
Next
SELECT (612.02µs)
SELECT (490.03µs)
HTTP GET (2.73ms)
GET /api/ingredients/mozzarella (1.89ms)
SELECT (602.59µs)
SELECT (478.61µs)
SELECT (509.92µs)
HTTP GET (2.68ms)
GET /api/ingredients/topping (1.85ms)
SELECT (578.05µs)
SELECT (428.92µs)
SELECT (636.78µs)
HTTP GET (2.69ms)
GET /api/tools (2.01ms)
SELECT (726.9µs)
SELECT (497.77µs)
SELECT (330.04µs)
HTTP GET (1.8ms)
GET /api/doughs (1.37ms)
SELECT (571.75µs)
SELECT (369.32µs)
SELECT (273.41µs)



1. What you are looking at :

This is **ONE trace** for:

```
POST /api/pizza   → 200 OK
```

But now, unlike before, this request:

- Calls **many internal HTTP endpoints**
- Runs **many SQL queries**
- Has **manual spans**
- Has **nested operations**

So you're seeing:

> 1 trace → 56 spans

---

2. The ROOT SPAN (top of the tree)

At the very top:

```
quickpizza: POST /api/pizza (68.5ms)
```

This is the **root span** because:

- It is the **entry point**
- It has **no parent**
- Everything else happens **inside it**

Think of it as:

> "The total time to create a pizza order"

---

3. Why you see SO MANY spans under it

Let's walk down the tree exactly as Grafana shows it.

HTTP child spans

Example:

```
HTTPPOST (1.66ms)
POST/api/users/token/authenticate (1.17ms)
```

This means:

- Your `/api/pizza` handler internally calls:
  - `/api/users/token/authenticate`
- OpenTelemetry auto-instrumented this HTTP call

👉 These are **child spans of the root span**

Database spans (SELECT / INSERT / COMMIT)

Example:

```
SELECT (613µs)
INSERT (595µs)
COMMIT (4.38ms)
```

These come from:

- Database auto-instrumentation
- Each query = **one span**

This is gold for debugging:

> "Which query is slow?"

Ingredient fetch spans

Example:

```
GET /api/ingredients/olive_oil
GET /api/ingredients/tomato
GET /api/ingredients/mozzarella
```

Each of these:

- Is an internal HTTP call
- Has its own DB queries under it

So the tree becomes:

```
POST/api/pizza
 ├──GET olive_oil
 │    └──SELECT
 ├──GET tomato
 │    └──SELECT
 └──GET mozzarella
      └──SELECT
```

Custom / manual span

This one stands out:

```
pizza-generation (37.8ms)
name-generation (37.7ms)
```

This is **not auto-instrumentation**.

This is:

- A **manual span**
- Probably wrapping business logic
- The **slowest part of the request**

👉 This is exactly why manual spans matter.

Internal recommendation service call

```
POST/api/internal/recommendations (9.26ms)
```

With:

```
BEGIN
INSERT
COMMIT
DELETE
```

This means:

- A transaction happened
- Multiple DB operations inside it

---

4. Why this is NOT visible in Root spans

This is the most important conceptual part 👇

What Root spans mode does

When you select **Root spans** in Grafana:

- Grafana shows **ONLY spans with no parent**
- It **hides all child spans**
- It aggregates by:
  - Endpoint
  - Service
  - Duration
  - Error rate

So for this trace, Root spans shows only:

```
POST /api/pizza
```

Because:

- That is the **only span without a parent**

Everything else has a parent → **hidden**

What All spans mode does

When you select **All spans**:

- Grafana includes:
  - Root span
  - Every child span
  - Every DB span
- You can:
  - Search `SELECT`
  - Filter `name-generation`

- Find slow internals

---

5. Simple mental model

Think of **Root spans vs All spans** like this:

Root spans

> "What requests are coming in and how long do they take?"

All spans

> "What exactly happened inside this request?"

---

6. Why Grafana defaults to Root spans

Because in production:

- You might have **millions of spans**
- But only **thousands of requests**

Root spans:

- Are cheaper to query
- Are easier to reason about at high level
- Are perfect for dashboards

All spans:

- Are for **debugging**, not dashboards

This trace is actually PERFECT

This trace shows **everything done right**:

✅ Entry span

✅ Internal HTTP calls

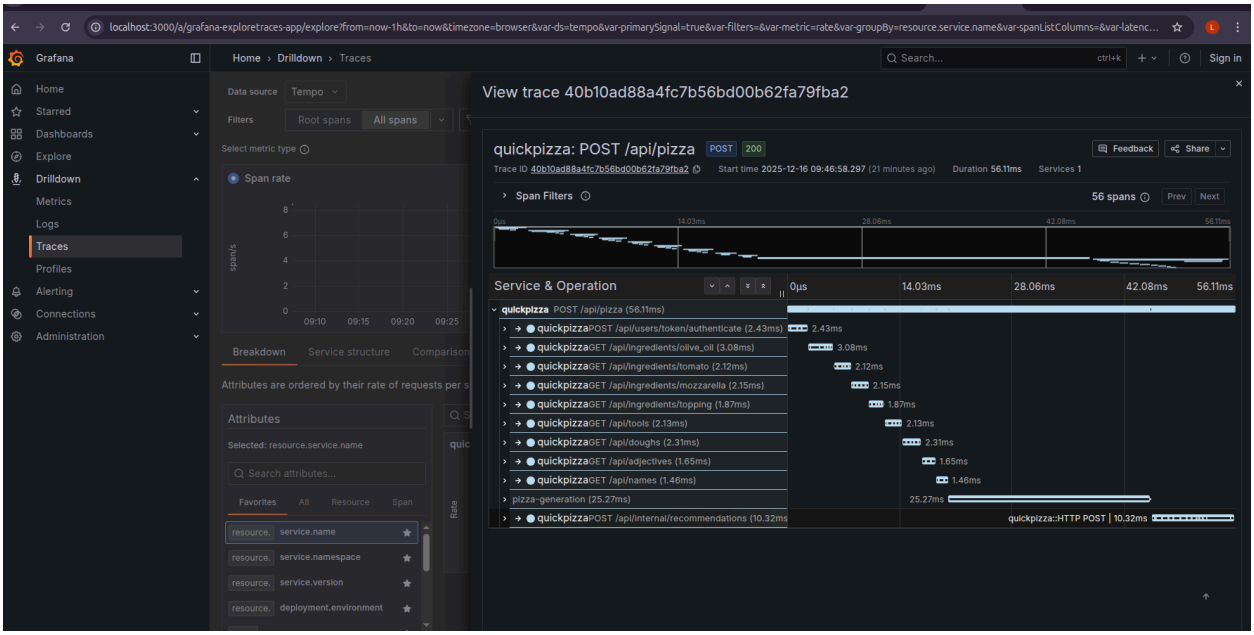
✅ DB auto-instrumentation

✅ Manual business spans

✅ Proper parent–child relationships

This is **exactly** what real-world production traces look like.

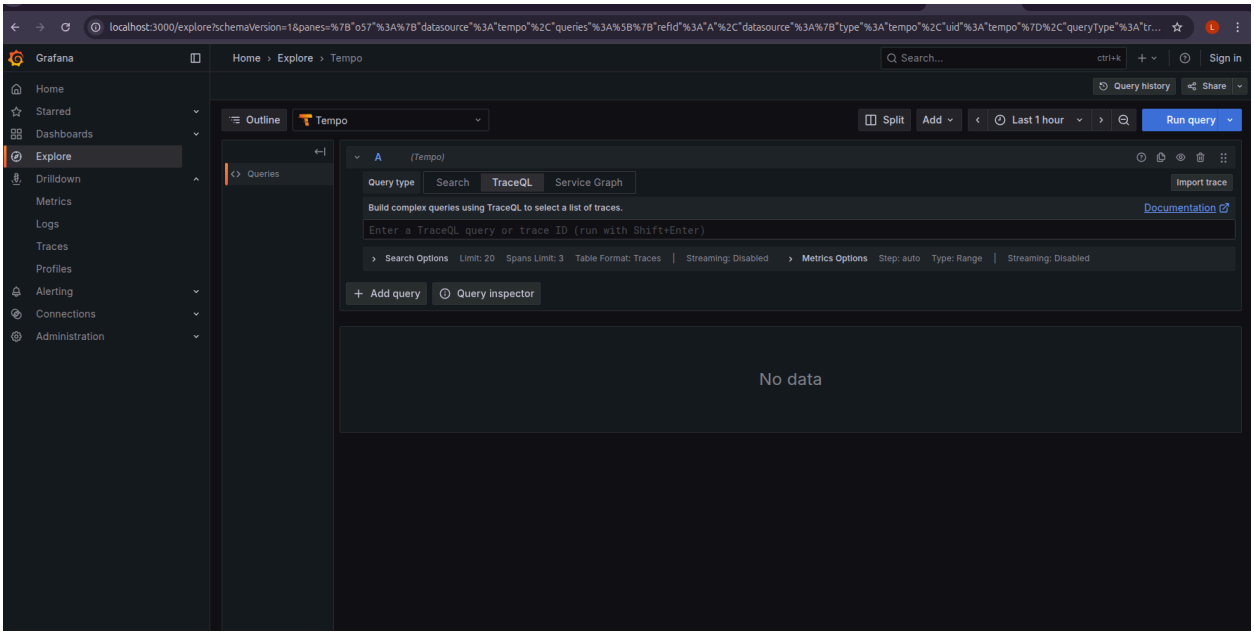
If you comapact the traces this is how it looks like and you can see the time it spent in the request.



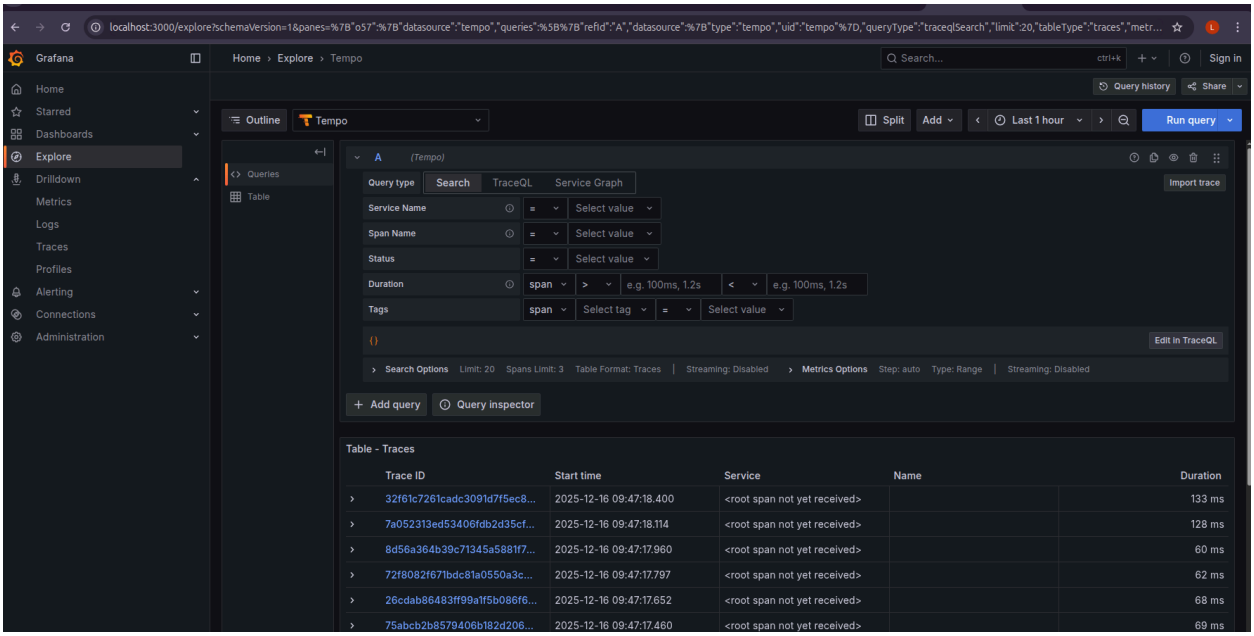
Now i have reloaded the site in the browser and have added more traffic by clicking the “pizza,please!” button.

Lets go to the explore menu and check our traces there.

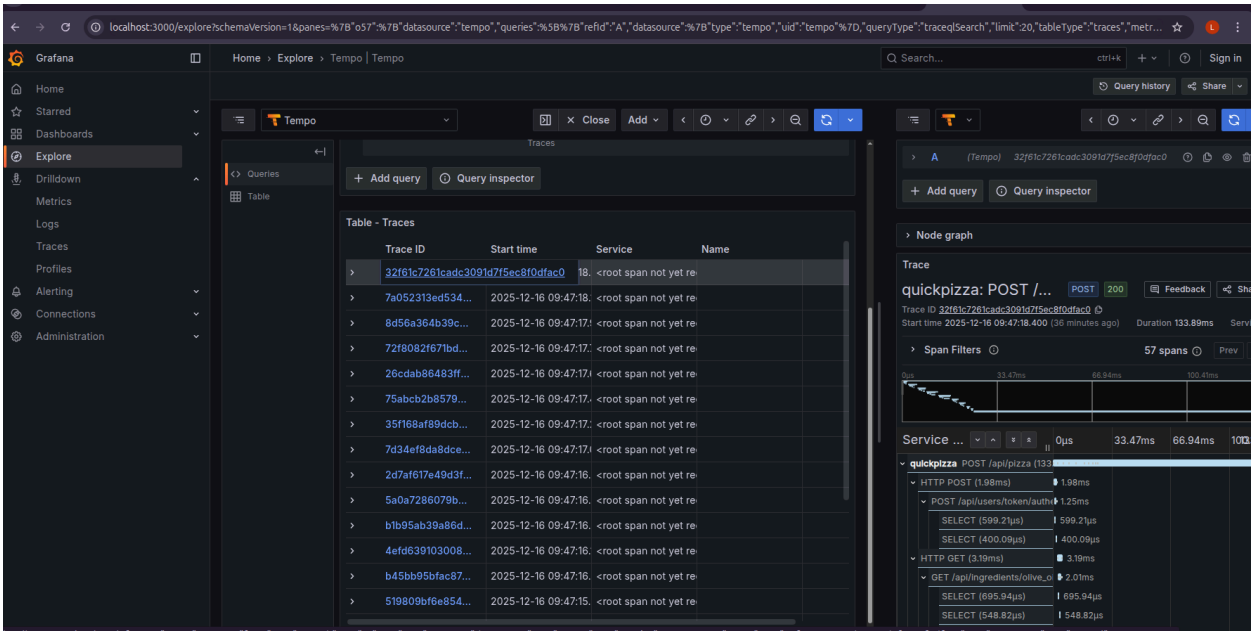
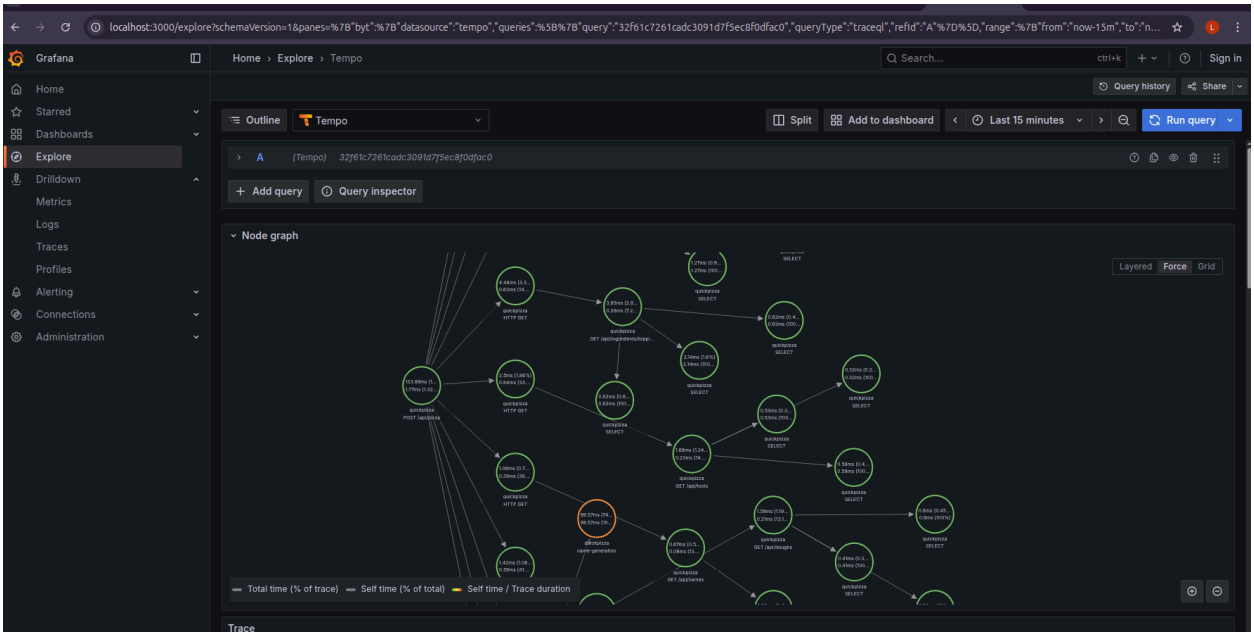
Make sure you select outline as tempo.



Query type select as “search”



You can see all the traces available here.



You can select as node graph to understand it better.